Inference Engines for LLMs & Local AI Hardware (2026 Edition)

You don't pick an inference engine first. You pick a hardware strategy, a workload shape, and a serving model. The engine follows.

That is the most useful way to think about LLM inference engines.

Series note: This is Part 3 in my series teaching Self-hosted LLMs / Local AI.

- Part 1: **GPU Memory Math for LLMs (2026 Edition)**
  .
- Part 2: **Memory Bandwidth for Local AI Hardware (2026 Edition)**
  .

Those two pieces explain the hardware capacity and bandwidth math.

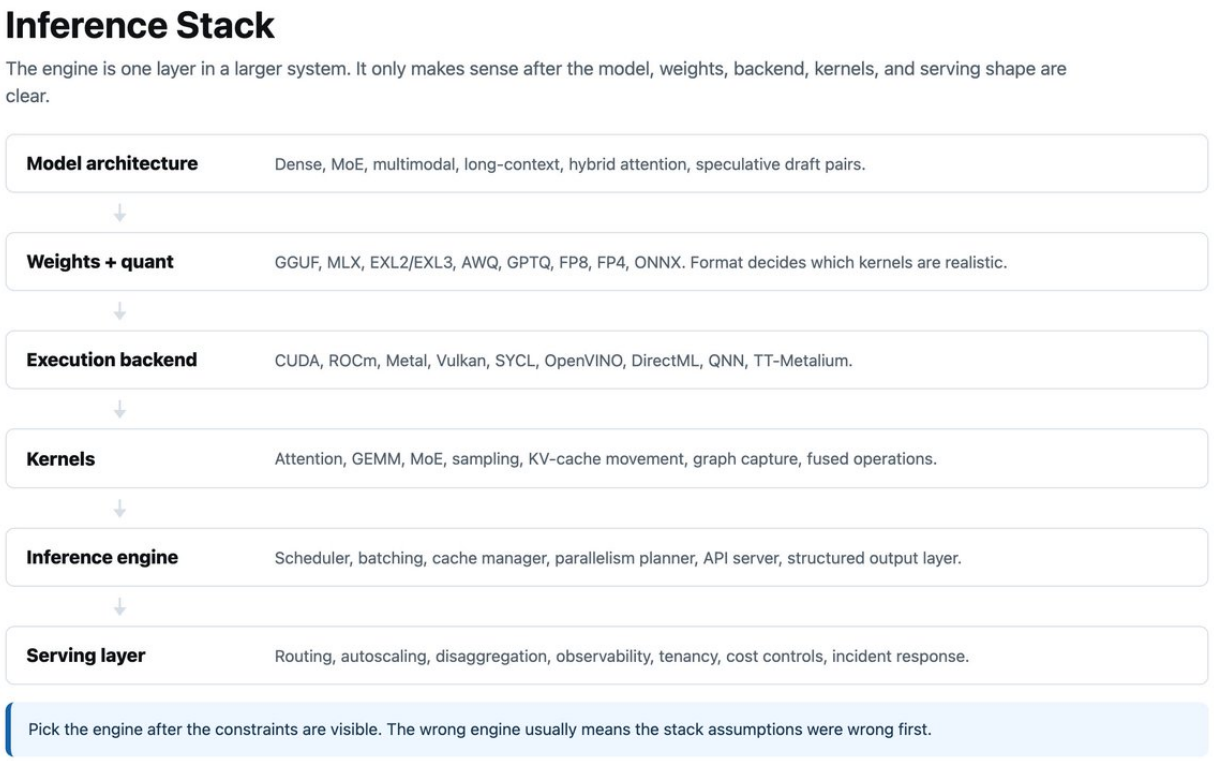
This one explains the software layer that turns that hardware into usable inference.

Engines

These tools serve different purposes / occupy different layers

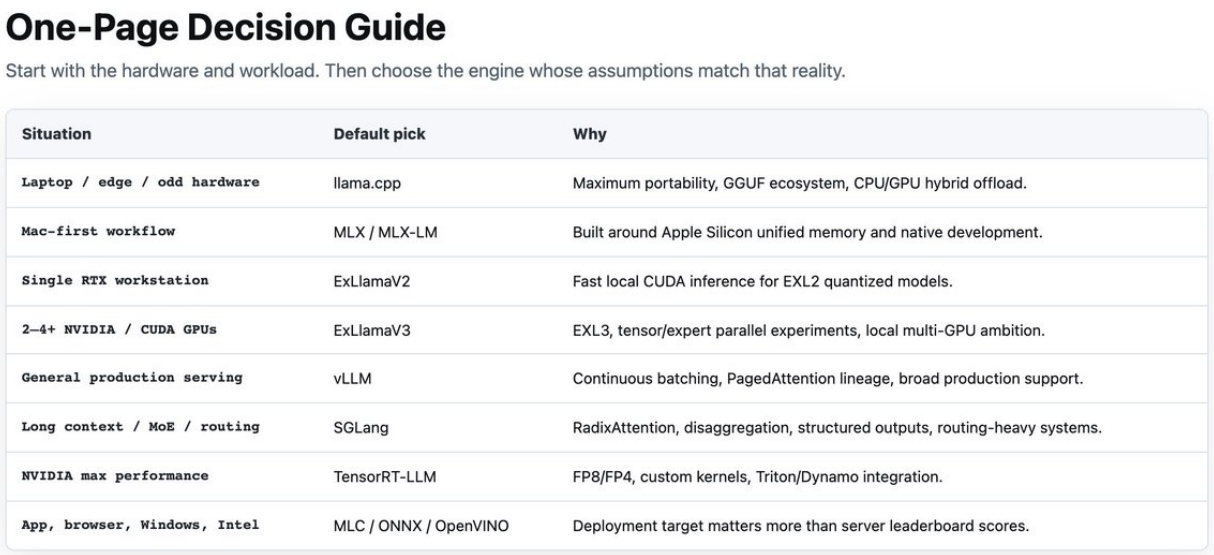
- Local portability
- Consumer CUDA
- Apple unified-memory workflows
- Quantized inference
- Production serving
- Distributed orchestration
- Vendor-optimized datacenter execution

A useful mental model:



The inference engine is not "the model." It is the traffic cop, memory manager, kernel dispatcher, scheduler, cache accountant, parallelism planner, API surface, and sometimes the deployment framework.
The best engine matches your **memory hierarchy**, **interconnect**, **quantization format**, **latency and throughput targets**, **model architecture**, and **operational maturity**.

The one-page decision guide



One-Page Decision Guide

Start with the hardware and workload. Then choose the engine whose assumptions match that reality.

Situation	Default pick	Why
Laptop / edge / odd hardware	llama.cpp	Maximum portability, GGUF ecosystem, CPU/GPU hybrid offload.
Mac-first workflow	MLX / MLX-LM	Built around Apple Silicon unified memory and native development.
Single RTX workstation	ExLlamaV2	Fast local CUDA inference for EXL2 quantized models.
2–4+ NVIDIA / CUDA GPUs	ExLlamaV3	EXL3, tensor/expert parallel experiments, local multi-GPU ambition.
General production serving	vLLM	Continuous batching, PagedAttention lineage, broad production support.
Long context / MoE / routing	SGLang	RadixAttention, disaggregation, structured outputs, routing-heavy systems.
NVIDIA max performance	TensorRT-LLM	FP8/FP4, custom kernels, Triton/Dynamo integration.
App, browser, Windows, Intel	MLC / ONNX / OpenVINO	Deployment target matters more than server leaderboard scores.

- **Laptop / edge / odd hardware** → llama.cpp
- **Mac-first workflows** → MLX / MLX-LM
- **Single RTX local inference** → ExLlamaV2
- **2-4+ NVIDIA / CUDA GPUs** → ExLlamaV3
- **General production serving** → vLLM
- **Long-context / MoE / routing** → SGLang
- **NVIDIA max performance** → TensorRT-LLM
- **Cluster orchestration** → NVIDIA Dynamo

The rest of this guide explains why.

What an inference engine actually does

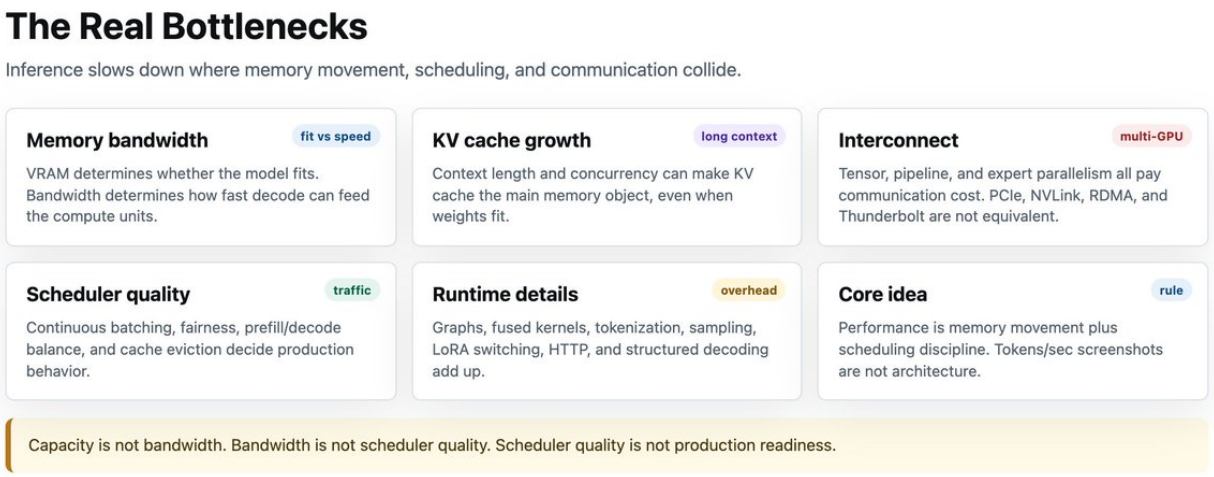
An inference engine loads weights, tokenizes input, runs the forward pass, samples tokens, maintains the KV cache, and streams results. Serious engines also handle batching, scheduling, prefix caching, quantization, parallel execution, API serving, metrics, and distributed execution.
The workload has two phases:
Prefill reads the prompt and builds the initial KV cache. It is compute-intensive.
Decode generates one token at a time, repeatedly reading weights and KV cache. It is memory-bandwidth-bound. Decode speed tracks memory bandwidth more than peak compute.
That distinction explains almost everything:

- **Short prompt, long answer:** decode dominates → memory bandwidth and batching matter.
- **Long prompt, short answer:** prefill dominates → attention kernels and chunked prefill matter.
- **Many users:** scheduler quality matters → continuous batching, cache paging, fairness.
- **Long context:** KV cache dominates → paged attention, KV quantization, offload.
- **MoE:** expert routing dominates → expert parallelism, interconnect, grouped GEMMs.

- **Multi-node:** interconnect dominates → NVLink, RDMA, pipeline parallelism, disaggregation.

PagedAttention tackled KV cache fragmentation. FlashAttention used IO-aware tiling to cut HBM (High Bandwidth Memory) traffic. Speculative decoding drafts cheap tokens and verifies them in parallel. The recurring theme: **inference performance is memory movement plus scheduling.**

The real bottlenecks



1. Memory bandwidth, not just VRAM size. VRAM determines fit. Bandwidth determines decode speed. Apple's M3 Ultra offers up to 819 GB/s unified-memory bandwidth. NVIDIA's H100 SXM lists 3.35 TB/s GPU memory bandwidth. Unified memory lets you **fit** models that would not fit in consumer VRAM. HBM lets you **serve** them faster when the model fits. Fit is not speed. Capacity is not bandwidth.

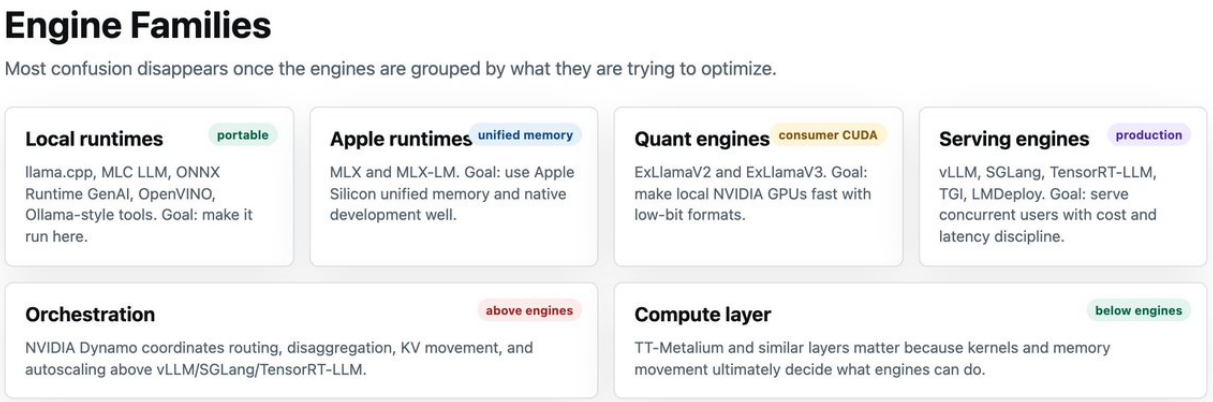
2. KV cache growth. KV cache grows with batch size and context length. Long-context workloads can run out of memory even when weights fit. PagedAttention partitions the KV cache into blocks, increasing utilization and supporting larger batches.

3. Interconnect. The moment a model crosses GPU boundaries (multi-GPUs), you pay communication cost. Tensor parallelism needs frequent all-reduce collectives. Pipeline parallelism communicates at stage boundaries. Expert parallelism needs all-to-all traffic for MoE. vLLM's docs note that without NVLink, pipeline parallelism can outperform tensor parallelism.

4. Scheduler quality. A good scheduler decides which requests enter the batch, how prefill and decode share the accelerator, whether long prompts block short decodes, and how to avoid starvation. Supporting batching is not the same as behaving like a production-ready scheduler.

5. Runtime overhead. CUDA graphs, kernel fusion, sampling overhead, tokenizer overhead, HTTP overhead, LoRA switching, and structured decoding all matter. At high scale, the annoying 2% overheads form a union and demand **attention** (no punt intended).

The engine families



There are four broad families:
Portable local runtimes: llama.cpp, MLC LLM, ONNX Runtime GenAI, OpenVINO, Ollama-style tools. These care about "make it run here."
Apple/unified-memory runtimes: MLX and MLX-LM. These care about "use big shared memory and Apple's stack well."
Consumer CUDA quant engines: ExLlamaV2 and ExLlamaV3. These care about "make my 3090/4090/5090 box scream with low-bit weights."
Production serving engines: vLLM, SGLang, TensorRT-LLM, TGI, LMDeploy. These care about concurrent users, KV cache, batching, parallelism, observability, and cost per token.
Then there are **orchestration layers** like Dynamo that sit above engines and coordinate fleets, disaggregated prefill/decode, routing, and autoscaling.

llama.cpp: the portability king

llama.cpp is the answer when the hardware is weird, constrained, offline, CPU-heavy, edge-oriented, or not a tidy NVIDIA datacenter node.
It supports Apple Silicon via ARM NEON, Accelerate, and Metal; x86 via AVX/AVX2/AVX512/AMX; RISC-V; low-bit quantization; CUDA; AMD via HIP; MUSA; Vulkan; SYCL; and CPU+GPU hybrid offload. That is why llama.cpp owns the "just make it run" lane.
The HTTP server is more capable than a "toy local runner". llama-server provides OpenAI-compatible routes, Anthropic Messages API compatibility, reranking, continuous batching, multimodal support, JSON schema constraints, function calling, speculative decoding, and a web UI.
The critical limitation: llama.cpp is not for serious multi-node production serving. Its RPC backend is explicitly documented as proof-of-concept, fragile, and insecure.
Verdict: Use llama.cpp when portability, offline operation, GGUF, or hybrid offload matter more than fleet-scale serving.
DO NOT use with
Multi-GPUs

MLX and MLX-LM: the Apple Silicon weapon

MLX is Apple's array framework for Apple Silicon, and MLX-LM is the LLM package built on it. It is a Mac-first ML stack.
The key hardware fact is unified memory. Apple Silicon gives the CPU and GPU direct access to the same memory pool. MLX arrays live in unified memory, and you choose the device when running the operation rather than moving arrays between separate memory spaces.
This changes the local inference tradeoff. On a discrete GPU system, the question is "does it fit in VRAM?" On an M-series Mac with large unified memory, the question becomes "does it fit in memory, and can the memory system feed the GPU fast enough?" Large quantized models can fit on machines where the same model would be impossible on a 24 GB consumer GPU.

However, it is also **slower**.
MLX-LM adds Hugging Face Hub integration, quantization, LoRA and full fine-tuning, distributed inference, and a large MLX Community model ecosystem. MLX is no longer Mac-only: it offers CUDA and CPU-only packages for Linux. Distributed communication supports MPI, Ring over TCP, JACCL for RDMA over Thunderbolt, and NCCL for CUDA.
MLX-LM's server itself warns that it is not recommended for production because it only implements basic security checks.
Verdict: Use MLX for Mac-first ML and LLM workflows. For high-concurrency public serving, start with a real serving stack.

ExLlamaV2 and V3: consumer CUDA, tuned and fast

ExLlamaV2 is the local CUDA quantization engine for people who want a consumer NVIDIA GPU to punch above its weight. It supports paged attention, dynamic batching, prompt caching, KV cache deduplication, batched generation, streaming, and speculative decoding. The word to remember is **local**. It makes quantized models fast on modern CUDA GPUs, especially consumer cards.
Best fits: one RTX 3090/4090/5090 box, local coding assistant, local chat, EXL2 quantized models, and prosumer workstation use.
ExLlamaV3 extends the philosophy toward multi-GPU and MoE-local inference. It adds the EXL3 quantization format based on QTIP, flexible tensor-parallel and expert-parallel inference for consumer hardware, an OpenAI-compatible server through TabbyAPI, continuous dynamic batching, and multimodal support.
V3 is compelling when you have 2-4+ consumer NVIDIA GPUs or want local MoE. Expect caveats: some models do not support tensor or expert parallelism in ExLlamaV3.
Verdict: ExLlamaV2 is the enthusiast's local CUDA engine. ExLlamaV3 is the frontier for multi-GPU (2-4) local setups. Expect rougher edges for better capability.

vLLM: the default open-source production server

vLLM is the first engine most teams should evaluate for serious opensource LLM serving.
It offers PagedAttention-based KV memory management, continuous batching, chunked prefill, prefix caching, CUDA/HIP graphs, extensive quantization (FP8, MXFP8/MXFP4, NVFP4, INT8, INT4, GPTQ, AWQ, GGUF), optimized attention and GEMM/MoE kernels, speculative decoding, torch.compile, and disaggregated prefill/decode/encode.
It is also flexible: tensor/pipeline/data/expert/context parallelism, streaming, structured outputs, tool calling, OpenAI-compatible and Anthropic Messages APIs, gRPC, multi-LoRA, and support for NVIDIA, AMD, x86/ARM/PowerPC CPUs, plus plugins for TPUs, Gaudi, Ascend, Apple Silicon, and more.
vLLM's docs note that multi-node deployments typically use Ray, and without NVLink, pipeline parallelism may beat tensor parallelism. The trap is assuming vLLM removes the need for systems thinking. You still need to tune batching, context length, GPU memory utilization, parallelism layout, and routing. vLLM gives you a very good engine; it still requires good System Design.
Verdict: If someone says "we need to serve open models in production," vLLM is the default starting point.

SGLang: vLLM's systems-brained cousin

SGLang is what you reach for when the serving workload is ugly: structured outputs, long context, MoE, disaggregation, and routing.
It offers RadixAttention prefix caching, prefill-decode disaggregation, speculative decoding, continuous batching, paged attention, tensor/pipeline/expert/data parallelism, structured outputs, chunked prefill, and multi-LoRA batching. It supports NVIDIA, AMD, Intel Xeon, Google TPUs, Ascend NPUs, and more.
SGLang's differentiator is serving architecture. Its prefill-decode disaggregation separates compute-intensive prefill from memory-intensive decode into specialized instances, transferring KV cache

between them. This prevents long prefill batches from interrupting decode and spiking token latency.
Verdict: SGLang is for teams whose bottleneck is no longer "can we run the model?" but "can we run it under hostile traffic without torching latency, memory, and cost?"

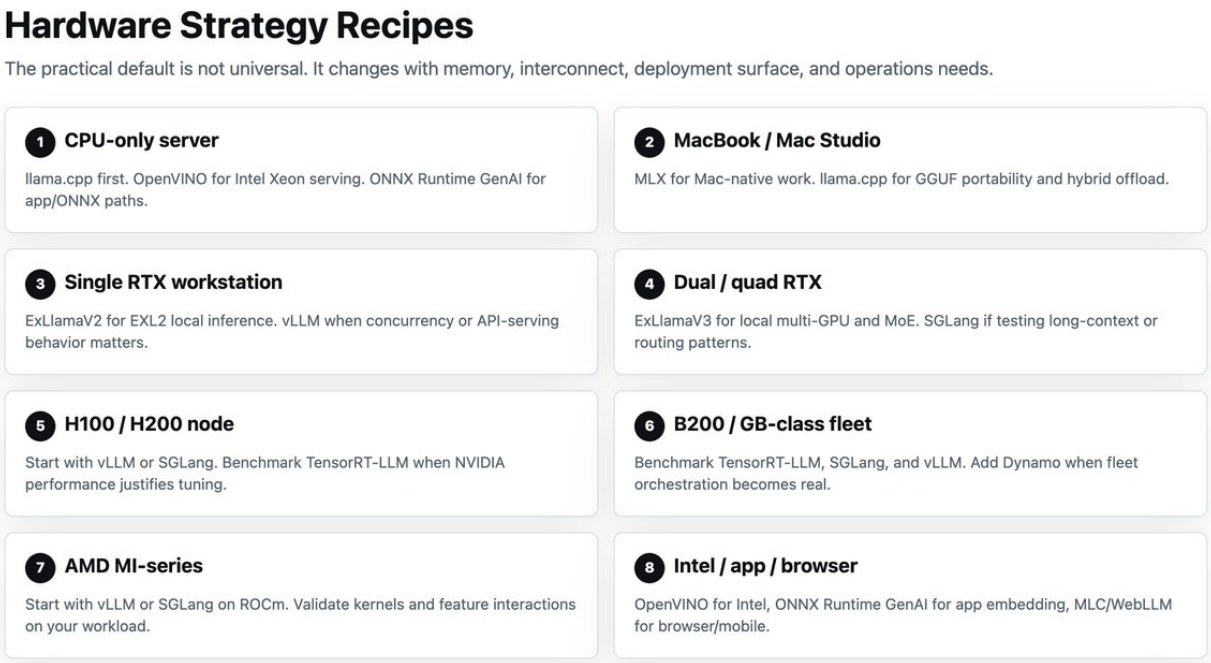
TensorRT-LLM: maximum NVIDIA performance

TensorRT-LLM is the NVIDIA-max-performance stack. It is optimized, specialized, powerful, and not pretending to be portable.
It provides Python APIs to build TensorRT engines with state-of-the-art optimizations, plus Python and C++ runtimes. It includes custom kernels for attention, GEMMs, and MoE; prefill-decode disaggregation, Wide Expert Parallelism, speculative decoding; and a high-level Python API integrated with NVIDIA Dynamo and Triton Inference Server.
B200 GPUs can load FP4 weights with optimized kernels. H100 and later support FP8 quantization that can double performance and halve memory consumption versus 16-bit with minimal accuracy loss.
Where it shines: H100/H200/B200/GB200/GB300-class fleets, NVIDIA-only datacenters, FP8/FP4 deployment, multi-node serving, and MoE at scale. Where it is awkward: AMD, Apple, or Intel portability; fast-changing experimental models; small local setups; and teams that need "works on everything."
Verdict: If you are committed to NVIDIA and care about absolute performance, TensorRT-LLM belongs in the bake-off. You trade portability for performance. Tuned specialization but less features.

The rest of the field

TGI is Hugging Face's production server with tracing, metrics, tensor parallelism, and continuous batching. Use it when HF integration and simplicity matter.
MLC LLM is the compiler-first universal deployment engine with OpenAI-compatible APIs across REST, Python, JavaScript, iOS, and Android. Best for "ship LLMs everywhere," especially browser, mobile, and native apps.
ONNX Runtime GenAI implements the full generative loop over ONNX Runtime and powers Foundry Local, Windows ML, and the VS Code AI Toolkit. It supports CPU, CUDA, DirectML, TensorRT-RTX, OpenVINO, QNN, WebGPU, and AMD GPU. Best for app deployment and ONNX workflows.
OpenVINO GenAI is the Intel-optimized story for Xeon CPUs, Arc GPUs, Core Ultra, and NPUs. It offers OpenAI-compatible serving with continuous batching and paged attention. Best for Intel hardware.
LMDeploy is a CUDA-focused toolkit with TurboMind for performance and PyTorch for accessibility. Most interesting for CUDA users who want an alternative to vLLM/SGLang/TensorRT-LLM.
NVIDIA Dynamo is a distributed orchestration layer above engines like vLLM, SGLang, and TensorRT-LLM, supporting disaggregation, intelligent routing, and multi-tier KV caching. Use it when single-engine serving is no longer enough.
Note: DO NOT USE Ollama.

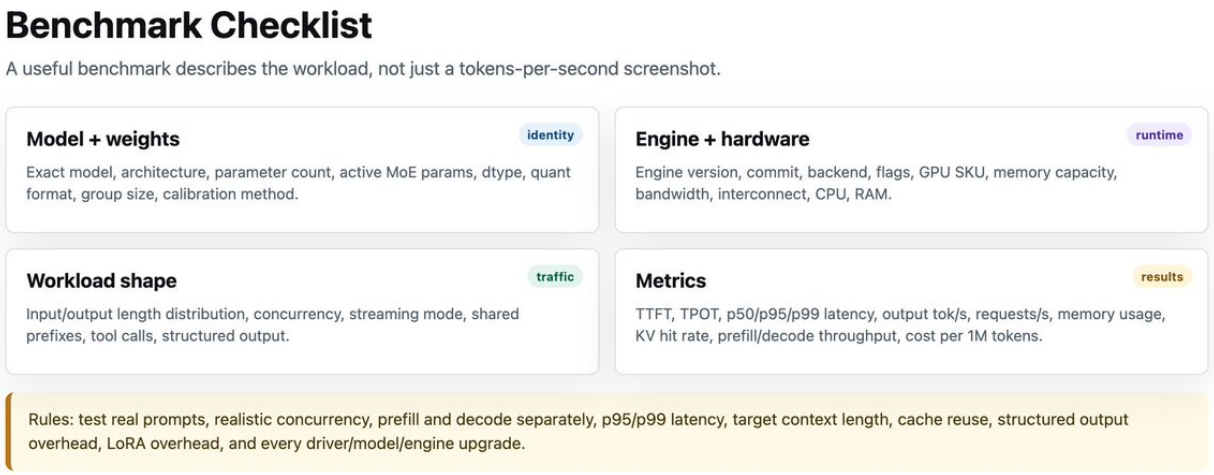
Hardware strategy recipes



CPU-only server: llama.cpp first. OpenVINO for Intel Xeon. ONNX Runtime GenAI for app/ONNX deployment.
MacBook / Mac Studio: MLX / MLX-LM for Mac-native workflows. llama.cpp for GGUF portability.
Single RTX 3090 / 4090 / 5090: ExLlamaV2 for EXL2 local inference. llama.cpp for GGUF or portability. vLLM if serving multiple users.
Dual or quad consumer RTX box: ExLlamaV3 for multi-GPU quantized inference or MoE. vLLM if serving behavior matters. SGLang if testing routing or long-context patterns.
8×H100 / H200 node: Start with vLLM or SGLang. Benchmark TensorRT-LLM if NVIDIA-only and performance justifies tuning. Use Dynamo when multi-node orchestration becomes necessary.
B200 / GB200 / GB300-class infrastructure: Benchmark TensorRT-LLM, SGLang, and vLLM. Add Dynamo for fleet-level orchestration, KV-aware routing, and autoscaling.
AMD MI300 / MI325 / MI350 / MI355: Start with vLLM or SGLang on ROCm. Avoid assuming NVIDIA benchmarks transfer cleanly.
Intel Xeon / Core Ultra / Arc: OpenVINO GenAI or OpenVINO Model Server. ONNX Runtime GenAI if app embedding matters.
Browser, mobile, app-native: MLC LLM / WebLLM or ONNX Runtime GenAI.

Benchmarking: what to measure

Bad benchmark: "I got 180 tok/s."



Good benchmark includes:
Model: exact model, architecture, parameter count, active MoE params.
Weights: dtype, quant format, group size, calibration.
Engine: version, commit, backend, flags.
Hardware: GPU SKU, memory capacity, bandwidth, interconnect, CPU, RAM.
Workload: input/output length distributions, concurrency, streaming, shared prefixes, structured output.
Metrics: TTFT, TPOT, end-to-end latency, p50/p95/p99, tokens per second, requests per second, GPU memory usage, KV cache hit rate, prefill throughput, decode throughput, cost per 1M tokens.
Benchmarking Rules:

1. Never compare engines using only single-user tokens per second.
2. Test your actual prompt and output distribution.
3. Test with realistic concurrency.
4. Separate prefill from decode.
5. Track p95 and p99, not only averages.
6. Measure memory headroom at target context length.
7. Test cache reuse if your app has repeated prefixes.
8. Benchmark structured output separately; grammar adds overhead.
9. Benchmark LoRA and multi-LoRA separately.
10. Re-test after driver, CUDA, ROCm, model, or engine upgrades.

Common mistakes

Choosing by VRAM capacity alone. VRAM determines fit. Bandwidth and scheduler determine speed. A large unified-memory machine can fit huge models, but an H100 decodes faster when the model fits due to much higher HBM bandwidth.
Using tensor parallelism on weak interconnect. Without NVLink or NVSwitch, test pipeline parallelism. vLLM's docs call this out for L40S-like setups.
Ignoring KV cache. Long context and concurrency can make KV cache the limiting factor. PagedAttention, prefix caching, KV quantization, and disaggregation are **not optional at scale**.
Treating local engines as production servers. llama.cpp server is capable. MLX-LM server is convenient. Ollama is pleasant yet SHOULD NOT BE USED.
However, **production means** security, observability, backpressure, routing, autoscaling, and SLA behavior. MLX-LM itself warns that its server is not recommended for production.
Assuming every quantization format is portable. GGUF, EXL2, EXL3, AWQ, GPTQ, FP8, FP4, MLX formats, and ONNX are not interchangeable. The right format is the one your engine has optimized kernels for.

Ignoring model architecture. Dense models, MoE, hybrid attention, multimodal models, and long-context variants stress different parts of the engine. Broad support does not mean every optimization works equally.
Trusting benchmark charts without workload shape. A chart for Llama 3.1 8B at 1K input / 128 output says little about a coding agent with 80K context running on Qwen 3.6 27B / Gemma 4 26B-A4B, or a RAG service with 500 concurrent users.

The opinionated final map

Local AI user: LM Studio or
Harbor
for convenience. llama.cpp for control. MLX on Mac. ExLlamaV2/V3 for CUDA local performance.
Building a local agent: Any should work, but given what most people use; llama.cpp for portability. MLX if users are on Apple Silicon. vLLM if simulating production serving locally.
Serving an internal team: Start with vLLM. Use SGLang if structured outputs, long context, multi-LoRA, MoE, or routing matter.
Serving customers at scale: Benchmark vLLM, SGLang, and TensorRT-LLM. If routing and disaggregation matter, SGLang and Dynamo deserve attention.
NVIDIA datacenter: TensorRT-LLM for max performance. vLLM for flexibility. SGLang for complex serving. Dynamo for fleet orchestration.
Apple Silicon: MLX for native development. llama.cpp for GGUF. Unified memory is a capacity superpower with bandwidth tradeoffs, not HBM.
Edge, app, browser, or Windows-native: llama.cpp, MLC LLM, ONNX Runtime GenAI, or OpenVINO, depending on stack.

Final principle

Inference Engines have consequences.
Pick the engine after answering these:

1. What hardware do I actually have?
2. Does the model fit in fast memory, or only in system/unified memory?
3. Is decode or prefill the bottleneck?
4. What context length and concurrency matter?
5. Are prompts shared enough for prefix caching?
6. Is the model dense, MoE, multimodal, or hybrid?
7. Do I need local convenience, production serving, or fleet orchestration?
8. What quantization format has optimized kernels on my target engine?
9. Is my interconnect PCIe, NVLink, NVSwitch, Ethernet, RDMA, or Thunderbolt?
10. Am I optimizing latency, throughput, cost, privacy, portability, or developer speed?

The engine follows the answers.